

CMPE 492
Stablecoin Flow Analysis on Blockchains

Celil Özkan

Advisor:
Prof. Can Özturan

June 10, 2026

Contents

1	INTRODUCTION	1
1.1	Broad Impact	1
1.2	Ethical Considerations	2
2	PROJECT DEFINITION AND PLANNING	3
2.1	Project Definition	3
2.2	Project Planning	4
2.2.1	Project Time and Resource Estimation	4
2.2.2	Success Criteria	5
2.2.3	Risk Analysis	5
2.2.4	Team Work (if applicable)	6
3	RELATED WORK	7
4	METHODOLOGY	9
5	REQUIREMENTS SPECIFICATION	12
6	DESIGN	16
6.1	Information Structure	16
6.2	Information Flow	17
6.3	System Design Diagrams	19
6.4	User Interface Design	20
7	IMPLEMENTATION AND TESTING	22
7.1	Implementation	22
7.2	Testing	23
7.3	Deployment	23
8	RESULTS	26
8.1	Data Collection	26
8.2	Address Classification Results	27

8.3	Graph Analysis and PageRank	27
8.4	Web Dashboard	28
9	CONCLUSION	29
A	COMMAND REFERENCE	32

Chapter 1

INTRODUCTION

1.1 Broad Impact

Stablecoins have become a critical infrastructure component in digital finance. They are widely used for cross-border settlement, centralized exchange (CEX) operations, decentralized exchange (DEX) liquidity provision, lending protocols, and cross-chain bridge transfers. As stablecoin activity spans multiple blockchain networks, understanding how value flows and which entities act as major hubs is important for both technical and economic analysis.

This project, named **ChainFlow**, builds a complete, end-to-end data pipeline and interactive dashboard for analyzing stablecoin transfer flows across EVM-compatible blockchains. Starting from raw on-chain event logs, the system collects transfer data, classifies addresses into entity types, constructs directed weighted graphs, computes PageRank-based influence scores, and exposes everything through an interactive web dashboard. A particular novelty is the inclusion of *x402 AI agent* classification – tracking autonomous payment-capable agents defined by the x402 protocol as a distinct entity class in the ecosystem.

Potential impact areas include:

- Measuring concentration of liquidity and systemic dependency on specific protocols.
- Identifying major transfer corridors between users, exchanges, bridges, and protocol contracts.
- Supporting transparent research on cross-network asset movement.
- Tracking the emergence of autonomous AI agents making on-chain stablecoin payments.

- Providing a practical and reproducible pipeline for blockchain data engineering and research.

Overall, ChainFlow contributes to improving the observability of decentralized financial systems, which is increasingly important as these systems continue to grow in scale and influence.

1.2 Ethical Considerations

This project uses only publicly available on-chain data, including transaction logs and metadata. No attempt is made to deanonymize blockchain addresses or associate them with real-world identities. Address classification is limited to high-level categories such as exchanges (CEX), bridges, and AI agents, without identifying individual persons.

Despite relying on public data, several ethical considerations arise:

- **Misclassification risk:** Address labels (e.g., CEX, Bridge) may be incomplete or outdated, leading to incorrect interpretations.
- **Attribution risk:** A single wallet address may represent multiple actors or automated systems, so behavioral conclusions must remain probabilistic rather than definitive.
- **Privacy considerations:** Although blockchain data is publicly accessible, large-scale aggregation and analysis can reveal behavioral patterns. Care is taken to avoid conclusions that could harm individuals or entities.
- **No personal identification:** The system does not attempt to infer or expose real-world identities behind addresses.
- **Method transparency:** All assumptions, filtering decisions, and limitations are clearly documented to ensure responsible interpretation of results.

To mitigate these risks, the project preserves exact chain and log identifiers, emphasizes reproducibility, and keeps analysis at the network and protocol level rather than focusing on individual actors.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

The objective is to build a modular, end-to-end system that collects stablecoin transfer events from multiple blockchains, classifies the participating addresses, constructs transaction flow graphs, and provides interactive tools for exploration.

Implemented scope:

- **Networks:** Ethereum, Polygon, Arbitrum, Avalanche, Optimism.
- **Tokens:** USDT, USDC, DAI (all networks), XAUT (Ethereum only).
- **Data source:** Infura JSON-RPC (`eth_getLogs`, `eth_blockNumber`).
- **Storage:** PostgreSQL 16 with partitioned `transfers` table, progress tracking, PageRank score/edge tables, and materialized views for aggregation.
- **Classification:** Four address classification categories applied to all stored transfers: (i) *CEX* – known centralized exchange hot-wallet addresses retrieved via per-network **Dune Analytics** queries (query IDs per chain); (ii) *Bridge* – bridge deposit and withdrawal contract addresses also retrieved from dedicated **Dune Analytics** queries and cached in per-network YAML files; (iii) *Mint/Burn* – detected directly from the transfer record when `from_address` or `to_address` equals the EVM zero address, requiring no external lookup; (iv) *x402 AI Agent* –

contract addresses registered as x402-enabled in the **8004scan.io** public agent registry, fetched via its paginated REST API and cached in per-network YAML files.

- **Graph analysis:** Directed weighted transfer graphs per network/token; PageRank computed using NetworkX and persisted to the database.
- **Dashboard:** FastAPI web application (ChainFlow) with three interactive tools: PageRank Explorer, Flow Explorer, and Ecosystem Overview.

2.2 Project Planning

The implementation was organized in three phases:

- (i) **Fetch Phase (complete):** Build and validate robust event extraction and database storage.
- (ii) **Analysis Phase (complete):** Address classification (CEX, Bridge, Mint/Burn, x402 agents) and graph construction with PageRank.
- (iii) **Reporting Phase (complete):** Interactive web dashboard, materialized views for aggregated analytics, and final documentation.

2.2.1 Project Time and Resource Estimation

Within the semester timeline, the main effort was distributed as follows:

- 20%: Architecture decisions (schema design, network/token selection, data source selection).
- 40%: Core implementation (fetcher, classification, graph engine, CLI command flow).
- 25%: Web dashboard and materialized view design.
- 15%: Validation, debugging, and documentation.

Main resources:

- Python ecosystem: `requests`, `click`, `psycopg2`, `pyyaml`, `dotenv`, `networkx`, `fastapi`, `uvicorn`.
- PostgreSQL 16 deployed via Docker.

- Infura RPC endpoints for blockchain event retrieval.
- Dune Analytics API for CEX address lists.
- 8004scan.io API for x402 AI agent registry.

2.2.2 Success Criteria

The project defines the following success criteria:

- Correctly fetch and decode ERC-20 **Transfer** logs for selected tokens.
- Support both automatic and manual block-range execution via CLI.
- Store events in a relational schema with conflict-safe insertion.
- Ensure resumability and traceability through chunk-level fetch progress records.
- Classify addresses into CEX, Bridge, Mint/Burn, and x402 agent categories.
- Compute and persist PageRank scores for each network and token combination.
- Serve an interactive web dashboard exposing graph, wallet, and ecosystem views.

All criteria have been met. The system holds over 3.4 million transfer events across five networks.

2.2.3 Risk Analysis

Key technical and project risks include:

- **RPC result cap (10k logs):** Mitigated through adaptive chunk resizing when the provider returns range overflow errors.
- **Rate limits / provider dependency:** Reduced by migrating from explorer APIs to Infura RPC and using controlled batch requests.
- **Data duplication:** Prevented via unique constraint (`tx_hash`, `log_index`, `network`) and `ON CONFLICT DO NOTHING`.

- **Classification coverage:** Addressed by combining Dune Analytics queries (CEX and Bridge), zero-address field inspection (Mint/Burn), and the 8004scan.io registry (x402 AI agents). Coverage is incomplete by design and treated as probabilistic.
- **Stale x402 address lists:** Mitigated by providing a CLI command to refresh the agent file from the live registry at any time.
- **Graph scale:** PageRank is pre-computed and stored in the database; only the top-10,000 scoring nodes are retained to keep query performance practical.

2.2.4 Team Work (if applicable)

The project is implemented by a single student developer. Progress was guided through regular advisor meetings and iterative feedback, with development organized into three milestones.

Chapter 3

RELATED WORK

This project is implementation-oriented and grounded in publicly available blockchain tooling and research. Four related areas are particularly relevant.

(1) Blockchain data providers and explorers. Explorer APIs such as Etherscan, Arbiscan, Polygonscan, Snowtrace, and Optimistic Etherscan [1, 4, 5, 7, 9] provide convenient access but become restrictive at high volume due to rate limits and endpoint constraints. JSON-RPC endpoints (e.g., Infura) provide lower-level access better aligned with event-centric pipelines. Practical RPC usage patterns are documented by Sharma [8] and the official Ethereum JSON-RPC specification [3].

(2) ERC-20 event-based analysis. Many token analytics workflows reconstruct transfer networks by parsing **Transfer** events rather than tracing full transaction internals. The ERC-20 standard is defined in the original Ethereum Improvement Proposal by Vogelsteller and Buterin [11]. Empirical research demonstrates that ERC-20 transaction networks can be analyzed effectively with graph methods for structural insights and prediction tasks [10].

(3) Graph centrality and PageRank. The PageRank algorithm, originally proposed by Page et al. [6] for ranking web pages by link importance, is directly applicable to blockchain address graphs where directed transfer edges carry weight proportional to token flow. This project uses NetworkX’s weighted PageRank implementation with a damping factor of 0.85 to rank addresses by structural influence in the transfer network.

(4) x402 protocol for autonomous AI agent payments. The x402 protocol [2] defines a machine-readable HTTP-based standard for AI agents to autonomously make stablecoin payments. Addresses associated with x402-enabled contracts are discoverable through the 8004scan.io registry. This project identifies such agents as a distinct entity class within the transfer ecosystem, enabling analysis of the emerging role of AI-driven financial ac-

tivity on public blockchains.

Compared to existing tools and approaches, ChainFlow focuses on building a transparent and reproducible pipeline that integrates data collection, classification, graph analytics, and interactive visualization in a single self-contained system.

Chapter 4

METHODOLOGY

The methodology is a four-phase data engineering and analysis pipeline designed for reproducibility and extensibility.

Step 1 – Configuration-driven design. Network metadata, RPC endpoints, token contract addresses, and decimal precision are stored in YAML configuration files (`networks.yaml` and `tokens.yaml`). CEX and bridge address lists are maintained in per-network YAML files. This design separates configuration from code and allows the pipeline to support additional networks or tokens with minimal code changes.

Step 2 – Command-line interface and modular pipeline. The pipeline exposes a CLI using `Click` with four implemented command groups:

- `fetch` – fetch transfer logs from the blockchain (automatic / manual modes)
- `classify_addresses` – label transfers by entity type (CEX, Bridge, Mint/Burn)
- `classify_agents` – identify x402 AI agent addresses and flag their transfers
- `graph` – build the transfer graph and compute PageRank scores

The FastAPI web server is launched separately and reads pre-computed results from the database.

Step 3 – Fetch logs with adaptive chunking. The fetcher retrieves `Transfer` events via Infura JSON-RPC (`eth_getLogs`). Adaptive block chunking adjusts the request size based on network activity and provider limits (maximum 10,000 logs per request). If a request exceeds the limit, the chunk size is halved and retried automatically. Fetch progress is stored in the `fetch_progress` table to support resumable execution and auditing.

Step 4 – Log decoding and normalization. Raw logs are decoded into structured records including block/transaction identifiers, timestamps, token metadata, sender and receiver addresses, and decimal-normalized transfer value.

Step 5 – Incremental persistence in PostgreSQL. Decoded records are written in batches to the `transfers` table using conflict-safe upsert operations. The table is partitioned by `network` and indexed across block number, address, and token fields for efficient temporal and entity-centric queries.

Step 6 – Address classification. Each network is processed by three classifiers:

- *Mint/Burn*: transfers where `from_address` or `to_address` is the EVM zero address (`0x000...0`) are labeled as minting or burning events.
- *CEX*: known centralized exchange hot-wallet addresses (sourced from Dune Analytics) are matched against `from_address/to_address` and labeled with `entity_class = 'CEX'`.
- *Bridge*: gateway and router contract addresses for cross-chain bridges (maintained in YAML config) are matched and labeled `entity_class = 'BRIDGE'`.

Labels are stored as `TEXT[]` arrays on each transfer row, allowing multi-label assignment where applicable.

Step 7 – x402 AI agent identification. Contract addresses registered as x402-enabled in the 8004scan.io registry are fetched via its public API. The agent address list is cached to a YAML file and then used to flag transfers originating from x402 agents (`is_from_x402 = true`). This boolean flag enables independent segmentation from the CEX/Bridge entity classes.

Step 8 – Transfer graph construction and PageRank. For each network (optionally filtered by token), aggregated transfer pairs are loaded from the database as a directed weighted edge list. A directed graph $G = (V, E)$ is built using NetworkX, where each edge (u, v, w) represents the aggregate stablecoin flow from address u to address v :

$$E = \{(u, v, w) \mid u = \text{from_address}, v = \text{to_address}, w = \sum \text{value}\}$$

Weighted PageRank is computed with damping factor $\alpha = 0.85$:

$$\text{PR}(v) = \frac{1 - \alpha}{|V|} + \alpha \sum_{u \in \text{in}(v)} \frac{\text{PR}(u) \cdot w(u, v)}{\sum_k w(u, k)}$$

The top 10,000 scored nodes and their connecting edges are persisted to the `pagerank_scores` and `pagerank_edges` tables.

Step 9 – Materialized views for aggregation. Four PostgreSQL materialized views pre-aggregate classification and x402 data for dashboard queries: (i) segment-level volume and transaction counts, (ii) x402 token breakdown, (iii) monthly x402 activity timeline, and (iv) top x402 agent leaderboard.

Step 10 – Interactive web dashboard. The ChainFlow dashboard is a FastAPI application serving three analysis tools:

- *PageRank Explorer:* a D3.js force-directed graph rendering the top- N addresses colored by entity class, with interactive zoom, node inspection, and network/token filters.
- *Flow Explorer:* a per-wallet view showing full inflow/outflow history, net balance, top counterparties by volume, and entity classification across networks and tokens.
- *Ecosystem Overview:* a network-level breakdown of stablecoin volume and transaction share by segment (CEX, Bridge, x402 Agent, Regular Wallet), with the x402 monthly activity timeline and top agent leaderboard.

Chapter 5

REQUIREMENTS SPECIFICATION

The system requirements are divided into functional and non-functional parts. Use case and activity diagrams are provided to visualize system interactions and the processing pipeline.

Functional Requirements

- (i) The system shall fetch ERC-20 **Transfer** logs for configured token contracts.
- (ii) The system shall support multiple EVM-compatible networks through a unified interface.
- (iii) The system shall provide both manual (user-defined block ranges) and automatic (resume from last processed block) fetching modes.
- (iv) The system shall decode and store transfer fields in a normalized PostgreSQL schema.
- (v) The system shall prevent duplicate inserts using deterministic uniqueness constraints (**tx_hash**, **log_index**, **network**).
- (vi) The system shall store fetch-progress checkpoints for each processed block chunk to support resumable runs.
- (vii) The system shall classify transfer addresses into CEX, Bridge, and Mint/Burn categories.

- (viii) The system shall identify x402 AI agent addresses and flag associated transfers.
- (ix) The system shall construct a directed weighted transfer graph per network/token and compute PageRank influence scores.
- (x) The system shall serve an interactive web dashboard for graph exploration, wallet flow analysis, and ecosystem overview.

Non-Functional Requirements

- (i) **Reliability:** Interrupted or failed runs should be recoverable using stored progress checkpoints.
- (ii) **Scalability:** Database schema and indexes should support growth across multiple chains and high-volume token events.
- (iii) **Maintainability:** Configuration files should allow easy addition or removal of chains, tokens, and address lists without modifying core code.
- (iv) **Traceability:** Each stored event should preserve source identifiers (`tx_hash`, `log_index`, `block_number`, `network`).
- (v) **Usability:** Command-line interface and web dashboard should be simple to operate.
- (vi) **Performance:** PageRank and ecosystem queries should be served from pre-computed database results to ensure sub-second dashboard response times.

Primary Use Cases

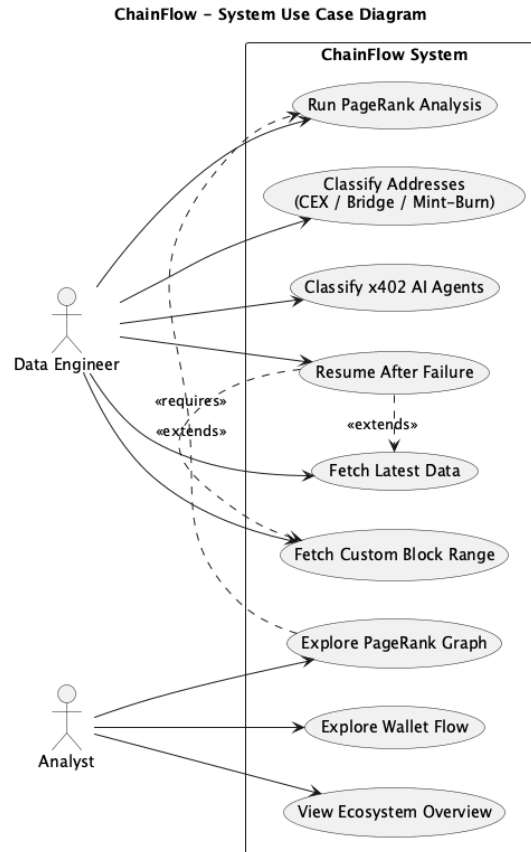


Figure 5.1: Use Case Diagram: ChainFlow System

- **UC-1 Fetch Latest Data:** Data engineer starts automatic mode; the system fetches from the last processed block to the latest chain block.
- **UC-2 Fetch Custom Block Range:** Data engineer provides start/end block; the system fetches the specified range.
- **UC-3 Resume After Failure:** Data engineer re-runs fetch; the system avoids duplicates and continues from remaining chunks.
- **UC-4 Classify Addresses:** Data engineer triggers address classification; the system applies CEX, Bridge, and Mint/Burn labels to all stored transfers.

- **UC-5 Classify x402 Agents:** Data engineer fetches the x402 agent registry and flags matching transfers in the database.
- **UC-6 Run PageRank Analysis:** Data engineer runs the graph command; the system builds the transfer graph and saves scored nodes and edges to the database.
- **UC-7–9 Dashboard tools:** Analyst accesses PageRank Explorer, Flow Explorer, or Ecosystem Overview via the web browser.

Chapter 6

DESIGN

6.1 Information Structure

The database schema consists of four main tables and four materialized views.

(1) **transfers** (partitioned by `network`)

- **Keys and provenance:** `tx_hash`, `log_index`, `block_number`, `block_hash`, `block_timestamp`.
- **Flow fields:** `from_address`, `to_address`, `value`, `raw_value`.
- **Asset context:** `token_symbol`, `token_address`, `network`.
- **Classification fields:** `from_entity_class` TEXT[], `to_entity_class` TEXT[], `is_from_x402` BOOL, `is_to_x402` BOOL, `event_class` VARCHAR(32).
- **Integrity:** unique constraint on (`tx_hash`, `log_index`, `network`).

(2) **fetch_progress**

- Stores processed chunk ranges and log counts per network.
- Unique on (`network`, `chunk_start`, `chunk_end`).

(3) **pagerank_scores**

- Stores pre-computed PageRank scores per (`network`, `token_symbol`, `address`).
- Indexed on (`network`, `token_symbol`, `score` DESC) for fast top- N queries.

(4) **pagerank_edges**

- Stores directed edges between the top-ranked nodes for graph visualization.
- Unique on (network, token_symbol, from_address, to_address).

Materialized Views: `mv_ecosystem_segments` (volume/count by entity segment), `mv_ecosystem_x402_tokens` (x402 token breakdown), `mv_ecosystem_x402_timeline` (monthly x402 activity), and `mv_ecosystem_x402_agents` (agent leaderboard) are pre-aggregated at pipeline run time to support sub-second dashboard queries.

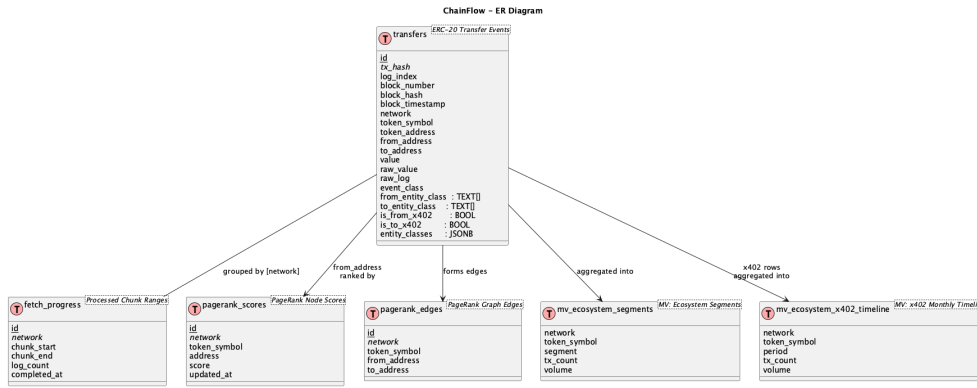


Figure 6.1: ER Diagram: ChainFlow Database Schema

6.2 Information Flow

The full data flow proceeds through four sequential phases, as illustrated in the activity diagram:

- (i) **Fetch:** CLI requests `eth_getLogs` for configured token addresses, decodes the results, and persists them incrementally to PostgreSQL.
- (ii) **Classify:** CLI runs address matching against CEX/Bridge/x402 label sources and updates entity class columns in the `transfers` table.
- (iii) **Graph:** CLI loads aggregated edges from the database, builds a directed weighted graph, computes weighted PageRank, and saves scores and edges back to the database.
- (iv) **Dashboard:** FastAPI reads pre-computed data from the database and serves interactive visualization pages to the browser.

Activity Diagram

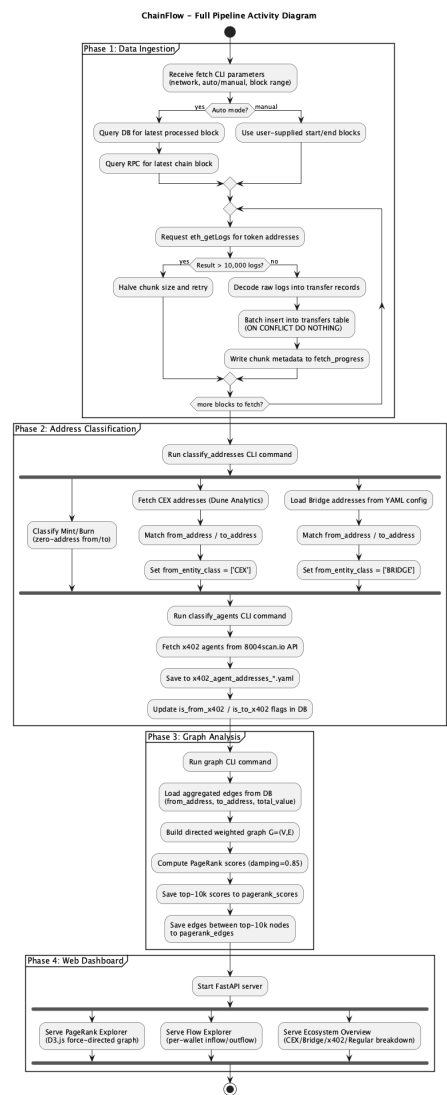


Figure 6.2: Activity Diagram: Full ChainFlow Pipeline

Sequence Diagram

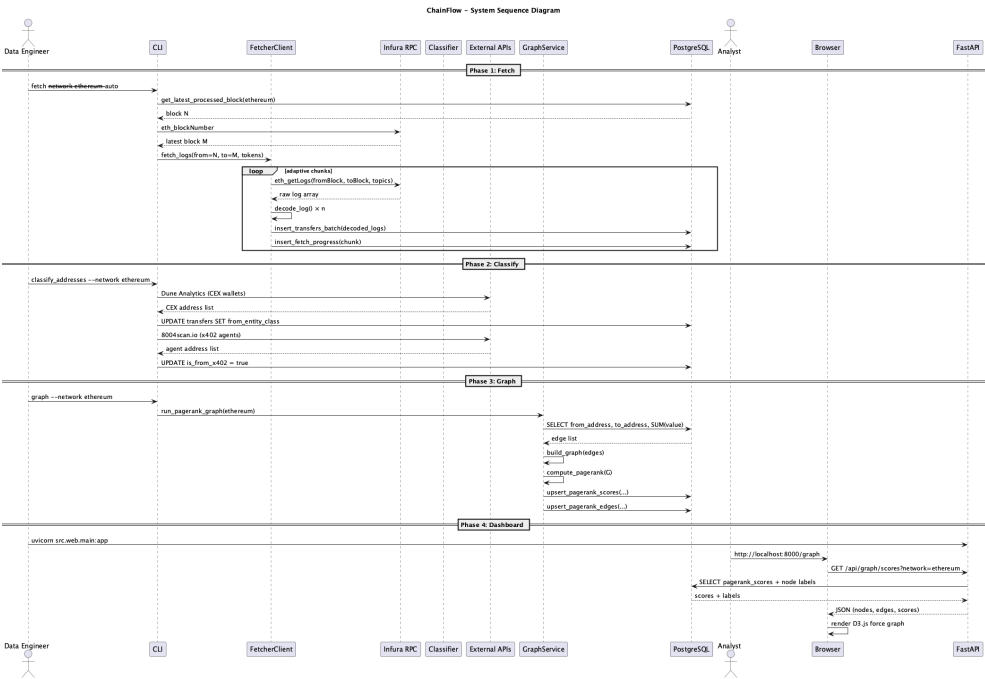


Figure 6.3: Sequence Diagram: End-to-End Data Flow from Fetch to Dashboard

6.3 System Design Diagrams

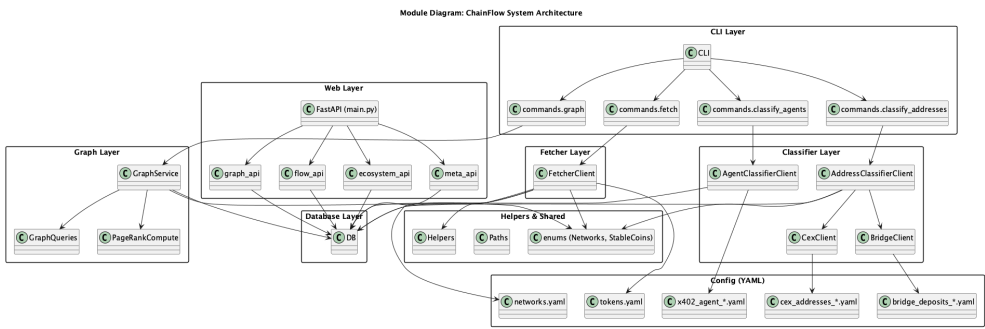


Figure 6.4: Module Diagram: ChainFlow Project Structure and Dependencies

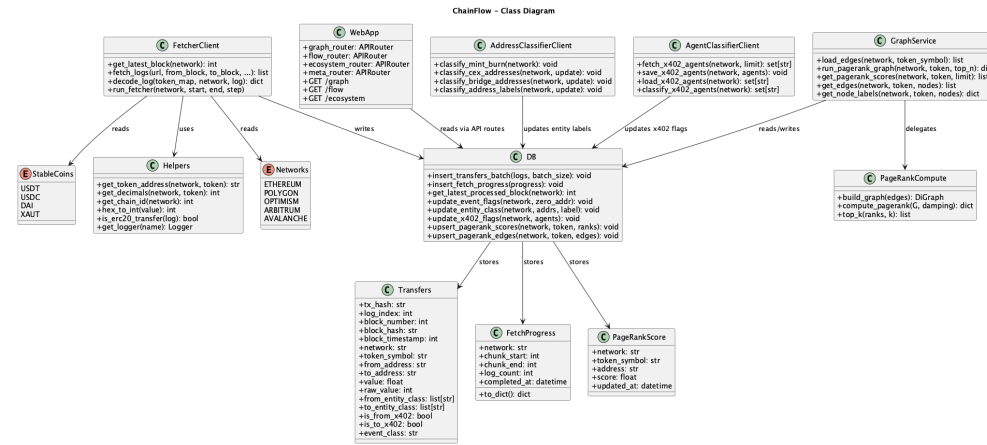


Figure 6.5: Class Diagram: Main Classes and Data Structures

Key design notes:

- **FetcherClient** handles RPC requests, log decoding, and adaptive chunking; it writes only to the database and has no dependency on any classifier or graph module.
- **AddressClassifierClient** is an orchestrator that delegates to **CexClient** and **BridgeClient**; it only issues **UPDATE** statements, never inserting rows.
- **AgentClassifierClient** handles the x402 classification path independently, including API pagination and YAML caching.
- **GraphService** reads edges from PostgreSQL, delegates computation to **PageRankCompute** (NetworkX), and writes results back to the database.
- The FastAPI web layer reads exclusively from pre-computed tables and materialized views, keeping read latency low and decoupled from pipeline execution.

6.4 User Interface Design

The current user interface has two components: a command-line interface for pipeline execution and a web dashboard for interactive analysis.

CLI: The Click-based CLI exposes four commands: **fetch**, **classify_addresses**, **classify_agents**, and **graph**. Each command accepts a **--network** parameter and additional flags appropriate to its function. The CLI is suitable

for scheduling (cron, cloud VM, CI jobs) and integrates cleanly with shell scripting.

ChainFlow Web Dashboard: The FastAPI application serves three views accessible from the browser:

- *PageRank Explorer (/graph)*: A D3.js force-directed graph showing the top- N addresses. Node size encodes PageRank score; color encodes entity type (CEX, Bridge, x402 Agent, or Regular Wallet). Users can filter by network and token, zoom/pan the graph, and click nodes to inspect address details.
- *Flow Explorer (/flow)*: A wallet-centric view. Users enter any address to see its complete transfer history, aggregate inflow and outflow per token, top counterparties by volume, and entity classification. Drill-down between connected wallets is supported.
- *Ecosystem Overview (/ecosystem)*: Network-level charts showing the share of total volume and transaction count attributable to CEX, Bridge, x402 Agent, and Regular Wallet senders. Additional panels show the x402 token breakdown and a monthly adoption timeline.

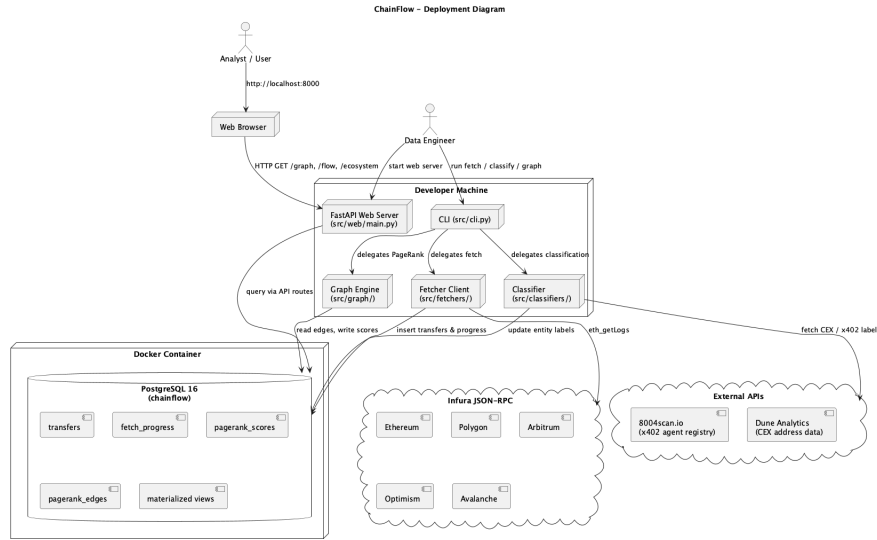


Figure 6.6: Deployment Diagram: ChainFlow System

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

The implemented pipeline uses Python with PostgreSQL 16 and a Dockerized database runtime. The web dashboard is served by Uvicorn/FastAPI.

Key implementation decisions:

- Switched from explorer APIs to Infura RPC for scalable, rate-limit-resilient event access.
- Implemented adaptive chunk resizing (halving on overflow) to handle high-density block ranges without manual intervention.
- Inserted transfer data in batches with `ON CONFLICT DO NOTHING` for idempotent re-runs.
- Partitioned the `transfers` table by `network` for better query organization and growth handling.
- Applied classification as an `UPDATE`-only pass over existing rows, keeping classification logic independent of ingestion.
- Used `TEXT[]` arrays for entity class columns to support multi-label assignment (e.g., an address may be both CEX and a bridge gateway).
- Cached x402 agent addresses in YAML files to decouple periodic registry refreshes from per-run classification.

- Used NetworkX for graph construction and PageRank. Only the top-10,000 scored nodes per network/token are persisted to keep storage and query overhead practical.
- Materialized views are refreshed via a dedicated API endpoint (`POST /api/ecosystem/refresh`) after each pipeline run.
- The D3.js force graph in the PageRank Explorer fetches only the top- N pre-ranked nodes and their connecting edges from the database, making the visualization independent of full graph traversal at render time.

7.2 Testing

Automated unit tests are not yet included in the repository. Validation was performed through manual and integration-style checks:

- CLI argument validation for all four command groups.
- Correct retrieval of latest block and latest processed DB block in auto mode.
- Successful decoding of ERC-20 **Transfer** event fields from raw log payloads.
- Duplicate-protection behavior through unique constraints and conflict handling.
- Correct insertion of chunk metadata into `fetch_progress`.
- End-to-end verification that CEX/Bridge/x402 flags are correctly applied to transfers.
- Sampling checks on PageRank output (verifying top addresses correspond to known exchange hot wallets).
- API response validation for all dashboard endpoints under different network/token filters.

7.3 Deployment

Deployment for development is lightweight, reproducible, and modular.

Environment Configuration

Create a `.env` file in the project root:

```
# PostgreSQL
POSTGRES_DB=chainflow
POSTGRES_USER=chainflowuser
POSTGRES_PASSWORD=chainflowpass
POSTGRES_PORT=5432

# Infura credentials
INFURA_API_KEY=your_infura_key
INFURA_API_SECRET=your_infura_secret

# 8004scan.io credentials (for x402 agents)
SCAN_API_KEY=your_scan_key

# Dune Analytics (for CEX addresses)
DUNE_API_KEY=your_dune_key
```

Database Setup

- (i) Start PostgreSQL container using Docker Compose:

```
docker compose up -d
```

- (ii) Execute SQL scripts to create schema, indexes, and materialized views:

```
psql -h localhost -U chainflowuser -d chainflow -f
sql/01_create_tables.sql
psql -h localhost -U chainflowuser -d chainflow -f
sql/02_create_indexes.sql
psql -h localhost -U chainflowuser -d chainflow -f
sql/11_ecosystem_views.sql
```

Running the Full Pipeline

- (i) Fetch transfer events:

```
python3 src/cli.py fetch --network ethereum --auto
```

- (ii) Classify addresses:

```
python3 src/cli.py classify_addresses --network ethereum
```

(iii) Classify x402 agents:

```
python3 src/cli.py classify_agents --network ethereum
```

(iv) Run PageRank:

```
python3 src/cli.py graph --network ethereum
```

(v) Start the web dashboard:

```
uvicorn src.web.main:app --host 0.0.0.0 --port 8000
```

Chapter 8

RESULTS

8.1 Data Collection

The ChainFlow database currently holds **3,466,862 transfer events** collected across five EVM-compatible networks. Table 8.1 shows the real distribution by network and token.

Table 8.1: Transfer event counts by network and token (actual database counts)

Network	USDT	USDC	DAI	XAUT	Total
Ethereum	643,605	503,879	13,256	3,560	1,164,300
Arbitrum	134,243	475,941	6,057	0	616,241
Polygon	191,104	85,371	328,392	0	604,867
Avalanche	144,699	386,929	30,642	0	562,270
Optimism	110,357	368,355	40,472	0	519,184
Total	1,224,008	1,820,475	418,819	3,560	3,466,862

Notable observations:

- USDC dominates across the dataset (52.5% of all events), driven by high activity on Arbitrum, Avalanche, and Optimism.
- DAI shows an unusual concentration on Polygon (328,392 out of 418,819 DAI events, or 78%), reflecting Polygon’s historically strong DAI ecosystem.
- Ethereum holds the largest single-network share (33.6%) with the most diverse token distribution including XAUT (gold-backed).

- The block range coverage spans: Ethereum blocks 24.6M–25.3M, Polygon/Avalanche blocks 10M–88M, Arbitrum blocks 100M–471M, Optimism blocks 100M–152M.

8.2 Address Classification Results

Table 8.2 summarizes the classification coverage across networks.

Table 8.2: Address classification results by network

Network	Total Tx	CEX Tx	Bridge Tx	x402 Tx	Regular
Ethereum	1,164,300	46,386 (4.0%)	44,939 (3.9%)	5,964 (0.5%)	~91.6%
Arbitrum	616,241	10,853 (1.8%)	28,127 (4.6%)	3,180 (0.5%)	~93.1%
Polygon	604,867	12,433 (2.1%)	11,830 (2.0%)	5,761 (1.0%)	~95.0%
Avalanche	562,270	7,166 (1.3%)	13,744 (2.4%)	5,743 (1.0%)	~95.3%
Optimism	519,184	5,261 (1.0%)	19,512 (3.8%)	3,691 (0.7%)	~94.5%

Key findings from classification:

- Ethereum has the highest absolute CEX and Bridge activity, consistent with its role as the primary settlement layer.
- Bridge transfers are proportionally higher on L2s (Arbitrum 4.6%, Optimism 3.8%), where cross-chain bridging is a primary on-ramp/off-ramp mechanism.
- x402 AI agent activity is present across all five networks (7–12 unique agent addresses per network), with Polygon and Avalanche showing the highest relative share (1.0%).

8.3 Graph Analysis and PageRank

PageRank was computed for all five networks and for each individual token, yielding up to 10,000 scored nodes per configuration. Table 8.3 shows the scored node counts.

Table 8.3: PageRank scored node counts by network (top-10k cap)

Network	All tokens	USDT	USDC	DAI	XAUT
Ethereum	10,000 (capped)	10,000	10,000	3,240	862
Arbitrum	10,000 (capped)	10,000	10,000	1,193	—
Avalanche	10,000 (capped)	10,000	9,832	3,350	—
Optimism	10,000 (capped)	10,000	9,797	1,950	—
Polygon	10,000 (capped)	—	10,000	—	—

The top-scoring address on Ethereum across all tokens (PageRank score ≈ 0.0128) corresponds to a well-known centralized exchange hot wallet, confirming that the graph correctly identifies high-volume hub addresses. The high cap-reaching on all major token/network combinations (most entries at exactly 10,000) indicates the transfer networks are well-connected with many structurally significant nodes.

8.4 Web Dashboard

The ChainFlow web dashboard is operational and accessible at `http://localhost:8000` after starting the FastAPI server. All three analysis tools – PageRank Explorer, Flow Explorer, and Ecosystem Overview – are functional and connected to the live database. Materialized views ensure that the Ecosystem Overview renders with sub-second response times regardless of database size.

Chapter 9

CONCLUSION

This project delivered ChainFlow, a complete end-to-end system for stablecoin flow analysis on EVM-compatible blockchains. The system progressed through all three planned phases:

- (i) **Fetch Phase:** A reliable, resumable ERC-20 event extraction pipeline collecting 3,466,862 transfer events across five networks and four tokens.
- (ii) **Analysis Phase:** Address classification (CEX, Bridge, Mint/Burn) and x402 AI agent identification, followed by directed transfer graph construction and weighted PageRank computation.
- (iii) **Reporting Phase:** The ChainFlow interactive dashboard, providing three views – PageRank Explorer, Flow Explorer, and Ecosystem Overview – backed by pre-computed database results and materialized views.

The project makes several concrete contributions:

- A scalable, transparent, and fully reproducible blockchain data collection pipeline.
- A labeled stablecoin transfer dataset covering five major EVM networks.
- A first-of-its-kind segmentation of x402 AI agent activity within the stablecoin transfer ecosystem.
- An interactive analysis dashboard that integrates graph theory, entity classification, and network-level statistics in a single self-contained system.

Remaining limitations:

- Automated unit tests are absent. Integration-style manual checks were used throughout.
- Classification coverage is incomplete by design; regular wallet addresses are classified by exclusion. CEX and Bridge label sources (Dune Analytics queries) may not be fully up-to-date.
- The block range collected for Ethereum (24.6M–25.3M, approximately 640k blocks) is relatively short compared to the chain’s history; longer windows would yield richer graph structure.
- The 10,000-node cap on stored PageRank results excludes a large fraction of addresses.

Future work directions:

- Add automated test coverage (unit tests for decode logic, integration tests for DB inserts).
- Expand Ethereum block range coverage; continue fetching across all networks.
- Improve address labeling by integrating additional classification sources (DeFiLlama, Arkham, on-chain contract introspection).
- Add temporal flow analysis: sliding-window PageRank and flow pattern tracking over time.
- Extend the dashboard with cross-chain flow comparison views and address cluster visualization.

Bibliography

- [1] Arbiscan. Arbiscan explorer. Online, 2026. <https://arbiscan.io/>, accessed 2026-03-29.
- [2] Coinbase Developer Platform. x402: Http payment protocol for ai agents. Online, 2025. <https://x402.org/>, accessed 2026-06-01.
- [3] Ethereum Foundation. Ethereum json-rpc api. Online, 2026. <https://ethereum.org/en/developers/docs/apis/json-rpc/>, accessed 2026-03-29.
- [4] Etherscan. Etherscan api documentation. Online, 2026. <https://docs.etherscan.io/introduction>, accessed 2026-03-29.
- [5] Optimistic Etherscan. Optimistic etherscan explorer. Online, 2026. <https://optimistic.etherscan.io/>, accessed 2026-03-29.
- [6] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, 1999.
- [7] Polygonscan. Polygonscan explorer. Online, 2026. <https://polygonscan.com/>, accessed 2026-03-29.
- [8] Tarun Kumar Sharma. A guide on ethereum rpcs, methods and calls. Blockchain Council, 2020.
- [9] Snowtrace. Snowtrace explorer. Online, 2026. <https://snowtrace.io/>, accessed 2026-03-29.
- [10] Sagi Somin, Gina Gordon, Alex Pentland, Erez Shmueli, and Yaniv Altshuler. Erc20 transactions over ethereum blockchain: Network analysis and predictions. *arXiv preprint arXiv:2004.08201*, 2020.
- [11] Fabian Vogelsteller and Vitalik Buterin. Erc-20: Token standard. Ethereum Improvement Proposal, 2015.

Appendix A

COMMAND REFERENCE

Data Ingestion

- Automatic fetch (from last DB block to chain tip):

```
python3 src/cli.py fetch --network ethereum --auto
```

- Manual fetch (specific block range):

```
python3 src/cli.py fetch --network polygon --manual  
--start 50000000 --end 50100000
```

Address Classification

- Classify CEX/Bridge/Mint-Burn addresses:

```
python3 src/cli.py classify_addresses --network ethereum
```

- Update x402 agent file from registry and classify:

```
python3 src/cli.py classify_agents --network ethereum  
--update
```

Graph Analysis

- Run PageRank for all tokens on a network:

```
python3 src/cli.py graph --network ethereum --save
```

- Run PageRank for a specific token:

```
python3 src/cli.py graph --network ethereum --token USDT
--top 20
```

Web Dashboard

- Start the FastAPI server:

```
uvicorn src.web.main:app --host 0.0.0.0 --port 8000
```

- Refresh materialized views after a pipeline run:

```
curl -X POST http://localhost:8000/api/ecosystem/refresh
```

Infrastructure

- Start PostgreSQL container:

```
docker compose up -d
```

- Run schema migration scripts:

```
psql -h localhost -U chainflowuser -d chainflow -f
sql/01_create_tables.sql
```